External Digital Twin Integration & Implementation Guide

Architecture: Dual-Mode Ingress (Real-Time WebSocket + Periodic REST Reconciliation)

Target Audience: Digital Twin Engineering Teams (Vizzio, Taylor, Proscalar, Unreal/Unity/Three.js Developers)
Gateway Entrypoint: Kong Gateway OSS 3.9 (HTTP/WS Port: `8088`, HTTPS/WSS Port: `8443`)
Protocols: REST (HTTP/1.1 JSON), WebSocket (RFC 6455 over Socket.IO / Engine.IO v4)
Reference Simulator: `twin_simulator.html`

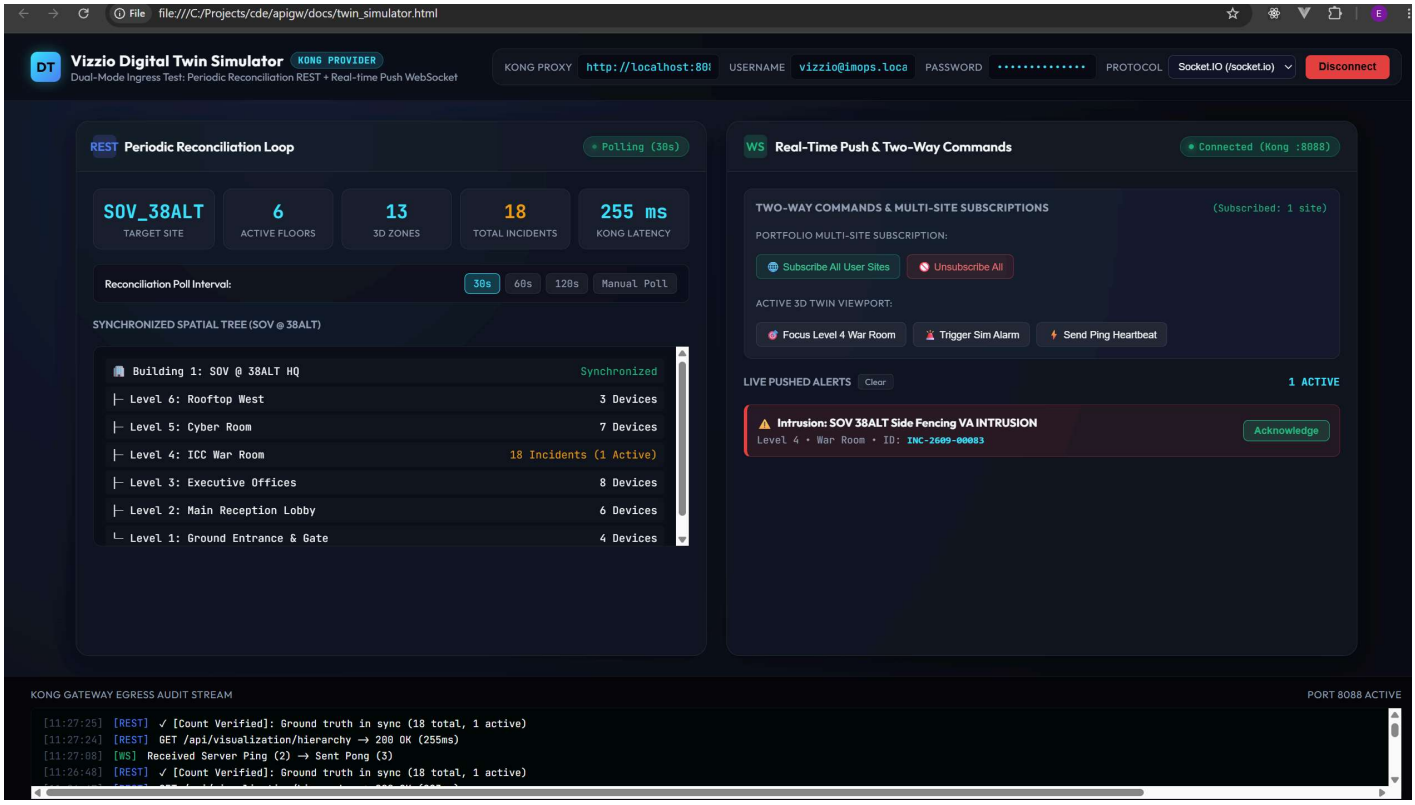
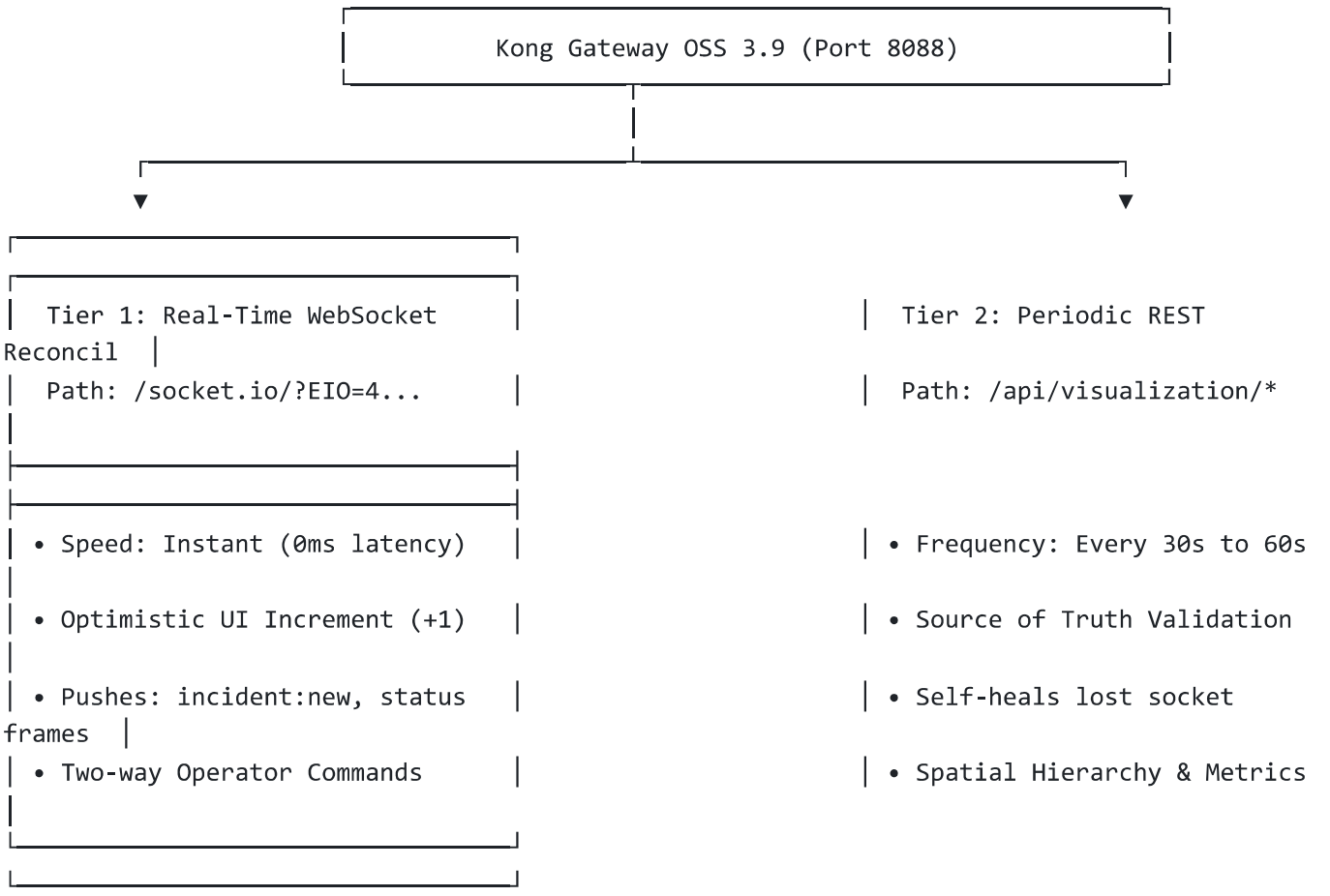


Figure 1: Reference Digital Twin Simulator (`twin_simulator.html`) demonstrating real-time WebSocket push ingress, periodic REST reconciliation (30s), live alert cards, and Kong Gateway egress audit logs.

1. Architecture Overview & Core Concept

To deliver high-fidelity, interactive 3D Digital Twins without data drift or network lag, integration follows a **Two-Tier Dual-Mode Pattern**:



- **Tier 1 (Streaming):** When cameras or sensors detect an intrusion or fire, Kong pushes the event across the open socket immediately. Your 3D scene renders the alarm and increments the count optimistically.
- **Tier 2 (Reconciliation):** Every 30–60 seconds, your client polls the REST endpoints to sync against the database. If a mobile tab was suspended or WiFi flickered, this reconciles the exact count and clears discrepancies.

2. Kong Gateway Routing & Upstream iMOPS Mapping Rules

Kong Gateway routes external Digital Twin requests to internal iMOPS backend services according to the following mapping definitions, path transformation rules, and perimeter security policies:

2.1 Route Configuration & Path Transformation Rules

Kong Route Name	Gateway Ingress Path	Strip Path (`strip_path`)	Target Kong Service	Target Upstream Back
`twin-visualization-route`	`/api/visualization`	`false`	`imops-twin-rest-service`	`http://<IMOPS_HOST>:`
`twin-auth-route`	`/api/auth`	`false`	`imops-twin-rest-service`	`http://<IMOPS_HOST>:`
`twin-ws-route`	`/ws`	`false`	`imops-twin-realtime-service`	`http://<IMOPS_HOST>:`
`twin-socketio-route`	`/socket.io`	`false`	`imops-twin-realtime-service`	`http://<IMOPS_HOST>:`

> [!IMPORTANT] **Preserved Path Prefix (`strip_path: false`):** Because `strip_path` is explicitly set to `false` , Kong Gateway does **not** truncate or modify the incoming URI prefix. Every request sent to `http://localhost:8088/api/visualization/<subpath>` forwards with a 1:1 path match directly to `http://<IMOPS_HOST>:13000/api/visualization/<subpath>` .

2.2 Complete Endpoint Mapping Directory (`/api/visualization/*`)

The `/api/visualization` ingress path exposes all spatial hierarchy, telemetry, device health, and incident reconciliation endpoints required by 3D Digital Twin runtimes:

Category	HTTP Method	Gateway Ingress URL (`Port 8088`)	Target Upstream iMOPS Endpoint (`Port 13000`)
Site Discovery	`GET`	`/api/visualization/hierarchy`	`/api/visualization/hierarchy`
Spatial Tree	`GET`	`/api/visualization/hierarchy`	`/api/visualization/hierarchy`
Site Buildings	`GET`	`/api/visualization/hierarchy`	`/api/visualization/hierarchy`
Building Floors	`GET`	`/api/visualization/hierarchy`	`/api/visualization/hierarchy`
Floor Zones	`GET`	`/api/visualization/hierarchy`	`/api/visualization/hierarchy`
Incident Sync	`GET`	`/api/visualization/incidents`	`/api/visualization/incidents`
Incident Search	`GET`	`/api/visualization/incidents`	`/api/visualization/incidents`
Incident Summary	`GET`	`/api/visualization/incidents`	`/api/visualization/incidents`

Category	HTTP Method	Gateway Ingress URL (`Port 8088`)	Target Upstream iMOPS Endpoint (`Port 13000`)
Equipment Inventory	`GET`	`/api/visualization/equipment`	`/api/visualization/equipment`
Device Health	`GET`	`/api/visualization/device-health`	`/api/visualization/device-health`
Visualization SLA	`GET`	`/api/visualization/sla`	`/api/visualization/sla`
Dashboard Telemetry	`GET`	`/api/visualization/dashboard`	`/api/visualization/dashboard`
Guard Attendance	`GET`	`/api/visualization/attendance`	`/api/visualization/attendance`
Security Summons	`GET`	`/api/visualization/summons`	`/api/visualization/summons`

Category	HTTP Method	Gateway Ingress URL (`Port 8088`)	Target Upstream iMOPS Endpoint (`Port 13000`)
VA Detection	`GET`	`/api/visualization/va-summary`	`/api/visualization/va-summary`
Body-Worn Camera	`GET`	`/api/visualization/bwc`	`/api/visualization/bwc`

2.3 Authentication & Header Forwarding Rules

1. **Perimeter Basic Auth Pass-Through (`hide_credentials: false`):**
    - Kong Gateway validates the client Basic Auth credentials against the `vizzio-twin-consumer` at the network edge.
    - Because `hide_credentials: false` is configured, Kong **preserves and forwards** the original `Authorization: Basic ...` header directly to the upstream iMOPS backend.
    - Upstream iMOPS services validate the credentials against their native authentication store without requiring secondary token exchange.
2. **Bearer Token Transparency:**
    - Clients utilizing JWT session tokens can supply `Authorization: Bearer <TOKEN>` . Kong passes the `Authorization` header downstream transparently to the iMOPS upstream.
3. **CORS & Preflight Handling:**
    - Preflight `OPTIONS` requests are handled and responded to directly by Kong Gateway (`Access-Control-Allow-Origin: *` , `Access-Control-Allow-Methods: GET, POST, OPTIONS` , `Access-Control-Allow-Headers: Authorization, Content-Type, apikey`).
4. **Traffic Shaping & Rate Limiting:**
    - Ingress requests are rate-limited to **120 requests/minute** per client to prevent spatial polling cycles from overwhelming upstream database queries.

3. Integration Workflow: Step-by-Step Sequence

Follow this exact order when initializing the Digital Twin engine:

```
Step 1: Authenticate Client Credentials
   |
Step 2: Fetch User Assigned Sites (REST)
   |
Step 3: Load 3D Spatial Hierarchy for Active Site (REST)
   |
Step 4: Establish Real-Time WebSocket Tunnel (WS)
   |
Step 5: Complete Handshake (0 -> 40) & Subscribe to Topics (42)
   |
Step 6: Listen for Inbound Real-Time Alerts & Handle Heartbeats (2 -> 3)
   |
Step 7: Start Background State Reconciliation Loop (Every 30s)
```

Step 1: Authentication & Credentials

Kong Gateway protects all endpoints via HTTP Basic Authentication and Consumer API Keys.

- **Gateway Base URL:** `http://<KONG_HOST>:8088` *(or `https://<KONG_HOST>:8443` in production)*
- **Username:** `vizzio@imops.local`
- **Password:** `xAJHkkm7m3V5MhtF0xGM`
- **Header Format:**

```
Authorization: Basic dml6emlvQGltb3BzLmxvY2FsOnhBSkhra203bTNWNU1odEYweEdN
```

Step 2: Fetch User Assigned Sites (Site Discovery)

Before loading a 3D building, discover which physical sites the authenticated user has access to.

- **Endpoint:** `GET /api/visualization/hierarchy`
- **Request:**

```
GET http://localhost:8088/api/visualization/hierarchy HTTP/1.1
Authorization: Basic dml6emlvQGltb3BzLmxvY2FsOnhBSkhra203bTNWNU1odEYweEdN
```

- **Response:**


```
{
  "success": true,
  "data": {
    "type": "sites",
    "items": [
      {
        "_id": "6a8289d3e9b4269e23c744d2",
        "name": "SOV @ 38ALT",
        "code": "SOV_38ALT",
        "status": "active"
      },
      {
        "_id": "6a7935e3ae5c46f4ab50a126",
        "name": "AGUSAN DEL NORTE- BUTUAN CITY- BAAN",
        "code": "ERP",
        "status": "active"
      }
    ],
    "pagination": { "totalCount": 1246 }
  }
}
```


- **Twin Action:** Populate the site selection menu or automatically target the user's primary assigned site (`SOV @ 38ALT`).

Step 3: Load Spatial Hierarchy & Construct 3D Scene

To render the 3D building model (floors, zones, and spatial coordinates), fetch the spatial hierarchy tree.

- **Endpoint:** `GET /api/visualization/hierarchy?siteName=SOV%20%40%2038ALT&tree=true`
- **Request:**

```
GET http://localhost:8088/api/visualization/hierarchy?
siteName=SOV%20%40%2038ALT&tree=true HTTP/1.1
Authorization: Basic dml6emlvQGltb3BzLmxvY2FsOnhBSkhra203bTNWNU1odEYweEdN
```

- **Key Fields Returned:**

```
{
  "success": true,
  "data": {
    "site": { "code": "SOV_38ALT", "name": "SOV @ 38ALT" },
    "totalFloors": 6,
    "totalZones": 13,
    "buildings": [
      {
        "name": "Building 1: SOV @ 38ALT HQ",
        "floors": [
          { "name": "Level 6: Rooftop West", "deviceCount": 3 },
          { "name": "Level 5: Cyber Room", "deviceCount": 7 },
          { "name": "Level 4: ICC War Room", "activeIncidents": 2 },
          { "name": "Level 1: Ground Entrance & Gate", "deviceCount": 4 }
        ]
      }
    ]
  }
}
```

- **Twin Action:** Build the 3D spatial scene hierarchy, attach zone colliders, and index each room/floor ID.

Step 4 & 5: Real-Time WebSocket Tunneling & Handshake

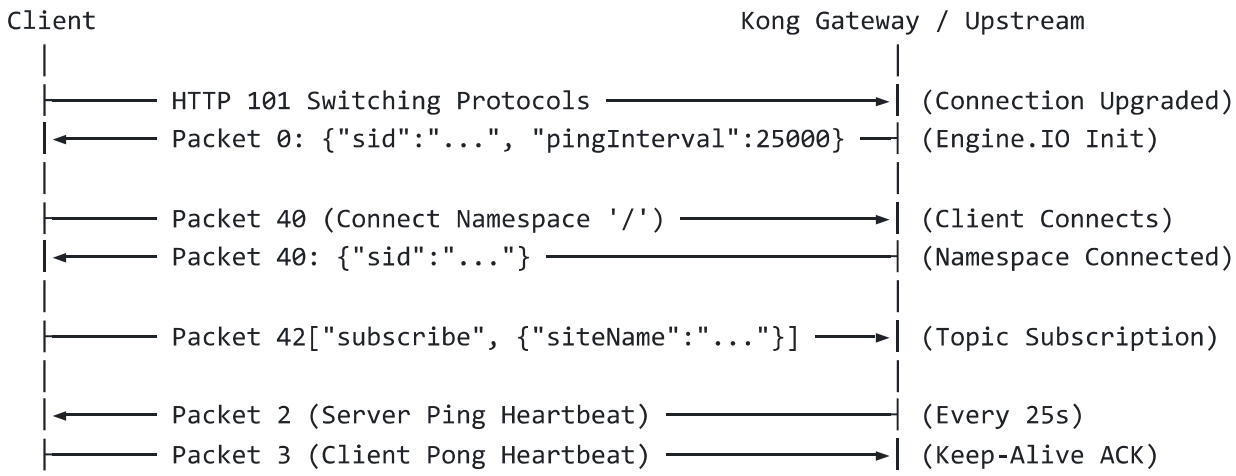
Establish the persistent bi-directional channel through Kong Gateway (`:8088`).

1. Connection URL

```
ws://<KONG_HOST>:8088/socket.io/?EIO=4&transport=websocket&apikey=vizzio-digital-twin-key-2026
```

2. Handshake Protocol Sequencing (Engine.IO v4)

> [!IMPORTANT] In Socket.IO / Engine.IO v4, **do not send event frames (`42`) immediately upon socket open**. You must strictly follow the `0` → `40` → `42` sequence:



3. Topic Subscription Payloads

A. Subscribe to Active 3D Site:

```
42["subscribe", {"siteName": "SOV @ 38ALT"}]
```

B. Subscribe to Fine-Grained Zone/Floor (When operator focuses Level 4):

```
42["subscribe", {"siteName": "SOV @ 38ALT:L4"}]
```

C. Batch Portfolio Subscription (All Assigned Sites):

```
42["subscribe_sites", {
  "sites": ["SOV @ 38ALT", "DORS", "SICC", "AIMS @ 1 Kallang Way 2A"]
}]
```

Step 6: Handling Inbound Real-Time Events

1. Event: `incident:new` (Incoming Alarm)

When an intrusion, fire, or crowding event is detected, Kong pushes this frame:

```
42["incident:new", {
  "incidentNumber": "INC-2609-00080",
  "site": "SOV @ 38ALT",
  "building": "HQ Building",
  "floor": "Level 4",
  "zone": "L4 Lift Lobby",
  "incidentType": "Loitering",
  "eventName": "Loitering: SOV 38ALT L4 LIFT LOBBY VA LOITERING",
  "priority": "medium",
  "status": "active",
  "detectedAt": "2026-09-17T03:05:29.160Z"
}]
```

Twin Actions on `incident:new`:

1. **Optimistic Counter Increment:** Immediately increment total and active incident counters (`+1`).
2. **3D Scene Highlighting:** Flash the affected zone/floor in red or amber (e.g. `Level 4: ICC War Room`).
3. **HUD Alert Card:** Spawn the alarm notification card on the operator's viewport.
4. **Camera Activation:** (Optional) Focus the 3D virtual camera onto the incident coordinates.

2. Event: `incident:updated` (Status Change)

Triggered when an incident is acknowledged or resolved by an operator:

```
42["incident:updated", {
  "incidentNumber": "INC-2609-00080",
  "site": "SOV @ 38ALT",
  "status": "acknowledged",
  "operator": "vizzio_twin_operator",
  "updatedAt": "2026-09-17T03:10:00.000Z"
}]
```

Twin Actions on `incident:updated`:

- Transition the 3D marker from pulsing red to acknowledged amber.
- Update UI card button to `Acknowledged ✓` .

Step 7: Two-Way Operator Commands (Twin → Gateway)

Digital Twin operators can perform 2-way actions directly from inside the 3D scene:

1. Acknowledge Incident

When the operator clicks "Acknowledge" on an alarm pin in 3D:

```
42["acknowledge_incident", {
  "incidentId": "INC-2609-00080",
  "siteName": "SOV @ 38ALT",
  "operator": "vizzio_twin_operator"
}]
```

2. Unsubscribe / Mute Topic Streams

When switching scenes or leaving the building:

```
42["unsubscribe", {"siteName": "SOV @ 38ALT"}]
```

Or pause all incoming streams:

```
42["unsubscribe_all", {}]
```

Step 8: Periodic State Reconciliation Loop (Self-Healing)

Every **30 to 60 seconds**, the Twin must run a background REST reconciliation to eliminate drift:

- **Endpoint:** `GET /api/visualization/incidents?siteName=SOV%20%40%2038ALT&daysBack=365&page=1&limit=10`
- **Response Payload:**

```
{
  "success": true,
  "data": {
    "summary": {
      "total": 14,
      "active": 1,
      "attend_now": 9,
      "resolved": 0
    },
    "items": [
      {
        "incidentNumber": "INC-2609-00080",
        "site": "SOV @ 38ALT",
        "status": "active",
        "floor": "Level 4"
      }
    ]
  }
}
```


Self-Healing Logic:

```
// Overwrite memory counter with database ground truth
const dbTotal = resJson.data.summary.total;
const dbActive = resJson.data.summary.active;

if (twinMemory.total !== dbTotal) {
  console.warn(`[Reconciler] Drift detected: memory had ${twinMemory.total}, DB has
${dbTotal}. Self-healing.`);
  twinMemory.total = dbTotal;
  twinMemory.active = dbActive;
  update3DSceneCounters(dbTotal, dbActive);
}
```

4. Drop-In Digital Twin Client SDK (JavaScript / TypeScript)

Here is the complete, production-tested client class implementing all the above steps:

```
/**
 * DigitalTwinGatewayClient.js
 * Production Client SDK for Kong Gateway Digital Twin Integration
 */
export class DigitalTwinGatewayClient {
  constructor(config) {
    this.kongHost = config.kongHost || 'http://localhost:8088';
    this.user = config.username || 'vizzio@imops.local';
    this.pass = config.password || 'xAJHkkm7m3V5MhtF0xGM';
    this.activeSite = config.siteName || 'SOV @ 38ALT';
    this.reconcilIntervalMs = config.reconcilIntervalMs || 30000;

    this.socket = null;
    this.totalIncidents = 0;
    this.activeIncidents = 0;
    this.subscribedSites = new Set();
    this.onAlertCallbacks = [];
  }

  getAuthHeader() {
    return 'Basic ' + btoa(`${this.user}:${this.pass}`);
  }

  // 1. Initial Site & Spatial Setup
  async initialize() {
    console.log(`[TwinClient] Initializing 3D Digital Twin for site: ${this.activeSite}`);

    // Fetch Spatial Tree
    const treeResp = await fetch(`${this.kongHost}/api/visualization/hierarchy?
siteName=${encodeURIComponent(this.activeSite)}&tree=true`, {
      headers: { 'Authorization': this.getAuthHeader() }
    });
    const treeData = await treeResp.json();

    // Initial Incident Sync
    await this.reconcileState();

    // Connect WebSocket
    this.connectWebSocket();

    // Start Periodic Reconciliation Timer
    setInterval(() => this.reconcileState(), this.reconcilIntervalMs);

    return treeData.data;
  }
```

```
  // 2. Real-Time WebSocket Channel
  connectWebSocket() {
    const wsUrl = this.kongHost.replace(/^http/, 'ws') + '/socket.io/?
EIO=4&transport=websocket';
    console.log(`[TwinClient] Connecting WebSocket: ${wsUrl}`);

    this.socket = new WebSocket(wsUrl);

    this.socket.onopen = () => {
      console.log('[TwinClient] WebSocket Channel Opened (101 Switching Protocols)');
    };

    this.socket.onmessage = (event) => {
      const raw = event.data;

      // Handle Engine.IO Ping/Pong Keep-Alive
      if (raw === '2') {
        this.socket.send('3'); // Respond with Pong
        return;
      }

      // Step 5A: Server sends Engine.IO Open -> Send Socket.IO Connect (40)
      if (raw.startsWith('0')) {
        this.socket.send('40');
        return;
      }

      // Step 5B: Socket.IO Connected -> Subscribe to Site Room
      if (raw.startsWith('40')) {
        this.subscribeToSite(this.activeSite);
        return;
      }

      // Step 6: Process Inbound Real-Time Events
      if (raw.startsWith('42')) {
        try {
          const parsed = JSON.parse(raw.slice(2));
          const [eventName, payload] = parsed;

          if (eventName === 'incident:new') {
            this.handleIncomingAlert(payload);
          } else if (eventName === 'incident:updated') {
            this.handleIncidentUpdated(payload);
          }
```

```
        } catch (err) {
          console.error('[TwinClient] Failed to parse frame:', raw);
        }
      }
    };

    this.socket.onclose = () => {
      console.warn('[TwinClient] Socket disconnected. Reconnecting in 5s...');
      setTimeout(() => this.connectWebSocket(), 5000);
    };
  }

  subscribeToSite(siteName) {
    if (this.socket && this.socket.readyState === WebSocket.OPEN) {
      this.socket.send(`42["subscribe",{"siteName":"${siteName}"}]`);
      this.subscribedSites.add(siteName);
      console.log(`[TwinClient] Subscribed to topic: site:${siteName}`);
    }
  }

  handleIncomingAlert(incident) {
    // 1. Optimistic Increment (+1)
    this.totalIncidents++;
    this.activeIncidents++;

    console.log(`[TwinClient] ⚡ LIVE ALERT: ${incident.incidentNumber} at
${incident.floor} ${incident.zone}`);

    // 2. Notify 3D Scene Handlers
    this.onAlertCallbacks.forEach(cb => cb(incident));
  }

  handleIncidentUpdated(update) {
    console.log(`[TwinClient] Incident ${update.incidentNumber} updated to:
${update.status}`);
  }

  // 3. Two-Way Operator Commands
  acknowledgeIncident(incidentId) {
    if (this.socket && this.socket.readyState === WebSocket.OPEN) {
      this.socket.send(`42["acknowledge_incident",
{"incidentId":"${incidentId}","operator":"${this.user}"}]`);
      console.log(`[TwinClient] Sent Acknowledged command for: ${incidentId}`);
    }
  }
```

```
  // 4. Periodic REST Reconciler (Self-Healing)
  async reconcileState() {
    try {
      const resp = await fetch(`${this.kongHost}/api/visualization/incidents?
siteName=${encodeURIComponent(this.activeSite)}&limit=10`, {
        headers: { 'Authorization': this.getAuthHeader() }
      });
      if (resp.ok) {
        const json = await resp.json();
        const dbTotal = json.data?.summary?.total ?? this.totalIncidents;
        const dbActive = json.data?.summary?.active ?? this.activeIncidents;

        if (this.totalIncidents !== dbTotal) {
          console.log(`[TwinClient] 🔄 Reconciled count: Syncing memory
(${this.totalIncidents}) -> DB Ground Truth (${dbTotal})`);
          this.totalIncidents = dbTotal;
          this.activeIncidents = dbActive;
        }
      }
    } catch (e) {
      console.error('[TwinClient] Reconciliation poll failed:', e.message);
    }
  }

  onAlert(callback) {
    this.onAlertCallbacks.push(callback);
  }
}
```

5. Verification & Testing Matrix

Use these quick tests to verify each step of the integration pipeline:

Step	Verification Command	Expected Output
Gateway Auth	`curl.exe -i -u "vizzio@imops.local:xAJHkkm7m3V5MhtF0xGM" http://localhost:8088/api/visualization/hierarchy`	`HTTP/1.1 200 OK` with JSON site list
WebSocket Upgrade	`curl.exe -i --max-time 3 -N -H "Connection: Upgrade" -H "Upgrade: websocket" -H "Sec-WebSocket-Version: 13" -H "Sec-WebSocket-Key: dGhl..." "http://localhost:8088/socket.io/?EIO=4&transport=websocket"`	`HTTP/1.1 101 Switching Protocols`
Trigger Live Incident	`curl.exe -i -u "vizzio@imops.local:xAJHkkm7m3V5MhtF0xGM" http://localhost:8088/va/sov-38alt-l4-icc-1-crowding`	`HTTP/1.1 200 OK` with `INC-2609-0008x` ticket
Interactive Simulator	Open `twin_simulator.html` in browser	Green `Connected (Kong :8088)` status

6. Visual Reference Breakdown: `twin_simulation.png`

The bundled simulator screenshot demonstrates the end-to-end integration running against live Kong Gateway (`:8088`):

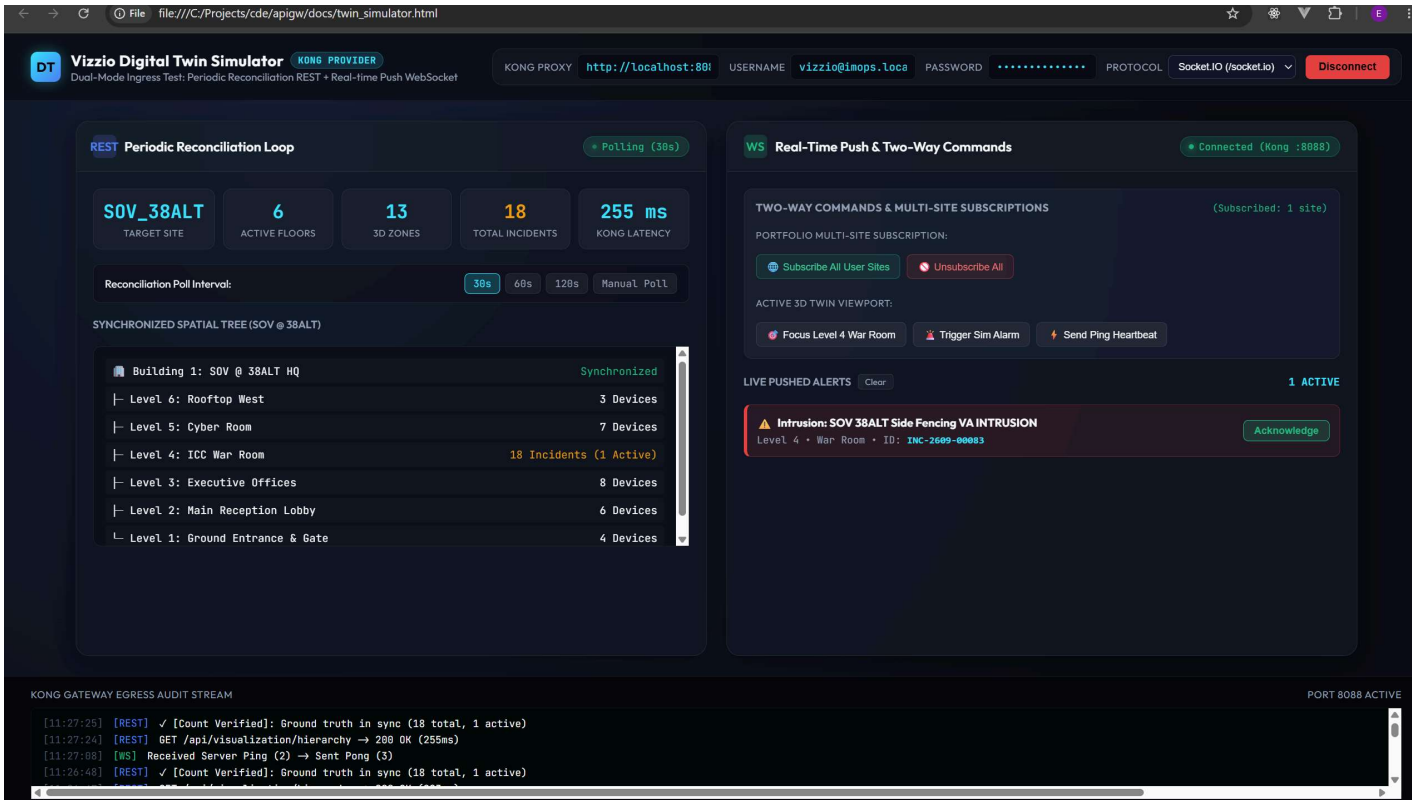


Figure 2: Component Breakdown of the Dual-Mode Digital Twin HUD.

Key UI Subsystems Mapped to 3D Twin Modules:

1. **Header & Connection Bar (Top):**
    - **Kong Proxy Host:** Configured to `http://localhost:8088`.
    - **Basic Auth Credentials:** Authenticates user `vizzio@imops.local` with Gateway-managed consumer key.
    - **Transport Protocol:** Engine.IO v4 over WebSocket (`/socket.io/?EIO=4&transport=websocket`).
2. **Left Panel — Tier 2: REST Periodic Reconciliation Loop:**
    - **Telemetry HUD Cards:** Tracks Active Site (`SOV_38ALT`), Active Floors (`6`), 3D Zones (`13`), Total Incidents (`18`), and Kong round-trip latency (`255ms`).
    - **Reconciliation Interval Selector:** Allows runtime toggling between 30s, 60s, 120s, or on-demand manual polling.
    - **Synchronized Spatial Tree:** Renders building hierarchy and highlights floors containing active alarms (`Level 4: ICC War Room - 18 Incidents (1 Active)`).
3. **Right Panel — Tier 1: Real-Time WebSocket Push & 2-Way Commands:**

    - **Status Badge:** Live connection indicator showing `• Connected (Kong :8088)`.
    - **Portfolio Multi-Site Subscription:** `Subscribe All User Sites` automatically discovers user site permissions and joins corresponding rooms (`subscribe_sites`).
    - **3D Viewport Controls:** Triggers virtual camera zoom (Level 4 War Room), test alarms, or keep-alive pings (`2 -> 3`).
    - **Live Pushed Alerts Stream:** Instant HUD notification cards displaying incident details (`INC-2609-00083`) with interactive 2-way `Acknowledge` action.
4. **Bottom Panel — Kong Gateway Egress Audit Stream:**
    - High-resolution log terminal displaying real-time WebSocket frames (`[WS]`) interleaved with background REST validation checks (`[REST]`).

7. Postman Resources & Artifacts

All accompanying testing collections and environment files are available in the repository:

- **Postman Collection:** `apigw/postman/SOV_38ALT_VA_Triggers.postman_collection.json`
- **Postman Environment:** `apigw/postman/iMOPS_Kong_Gateway.postman_environment.json`
- **Interactive Web Simulator:** `apigw/docs/twin_simulator.html`
- **Simulator Screenshot:** `apigw/docs/twin_simulation.png`